



Task2:

Evaluation Pipeline on the COMPAS Dataset

Project Report

Ahmed Abdullah Busaeed

1. Introduction

The COMPAS (Correctional Offender Management Profiling for Alternative Sanctions) dataset is commonly used in machine learning research to study algorithmic fairness. It contains criminal-history and demographic information used by a commercial tool for predicting the risk of reoffending. It also includes an outcome label, `two_year_recid`, which shows whether a person was re-arrested within two years.

The COMPAS tool has previously been reported to treat different demographic groups unequally, making it a common dataset for studying the balance between model performance and fairness in classification systems.

This report presents a structured Python project organized into separate modules rather than a single notebook. The project trains and compares different classification models on the COMPAS dataset, uses SHAP and LIME to explain the model predictions, and evaluates fairness based on two protected attributes: race and sex.

1.1 Objectives

- Build a modular, reusable pipeline (data loading → preprocessing → modelling → evaluation → explainability → fairness) instead of a single notebook.
- Train and compare at least three classification models on the COMPAS recidivism prediction task.
- Explain model predictions globally and locally using SHAP and LIME.
- Measure fairness differences across race and sex groups using standard error rate based metrics.
- Summarize the performance, explainability, and fairness results in a single report.

2. Project Structure

The project is organized as a Python package (`src/`), with a separate module for each stage of the pipeline. A `results/` folder is used to store trained models, figures, and explanation outputs. This structure allows each stage to be run and tested independently, making it different from a single, monolithic notebook.

File	Responsibility
<code>src/data_loader.py</code>	Loads <code>compas.csv</code> , validates that required columns are present, and reports dataset shape.
<code>src/preprocessing.py</code>	Selects features, encodes sex and race, and creates a stratified train/test split while keeping protected attributes aligned.
<code>src/models.py</code>	Trains Logistic Regression, Random Forest, and Gradient Boosting models, calculates accuracy and ROC-AUC, and saves the trained models in <code>results/</code> .
<code>src/evaluation.py</code>	Loads the saved models, evaluates their performance, and creates confusion matrix, ROC, and precision–recall plots for each model.
<code>src/explainability.py</code>	Generates a global SHAP summary plot for the overall model and a local LIME explanation for an individual test instance.
<code>src/fairness.py</code>	Computes FPR, FNR, precision, and recall by race and sex, reports max–min disparities, and creates fairness comparison charts.
<code>results/</code>	Stores trained models (<code>.pkl</code>), evaluation plots, SHAP/LIME outputs, and fairness charts from the modules in the project.

3. Dataset

The raw COMPAS dataset (compas.csv) used by the pipeline contains **7,214 rows and 53 columns**. It includes demographic information such as sex, race, age, and age category, criminal-history information such as priors_count, juv_fel_count, juv_misd_count, and juv_other_count; COMPAS risk score fields such as decile_score and score_text; and the target outcome, two_year_recid.

3.1 Feature Selection and Preprocessing

preprocessing.py limits the model input to six features and keeps sex and race for fairness analysis. Race is also one-hot encoded as a model feature.

- **Model features:** age, juv_fel_count, juv_misd_count, juv_other_count, priors_count, decile_score.
- **Target:** two_year_recid (1 = re-offended within two years, 0 = did not).
- **Protected attributes:** sex, race, retained for fairness evaluation.

Rows with missing values in the selected columns are removed. Sex is binary-encoded (Female = 0, Male = 1), while race is one-hot encoded. The data is split into **80% training and 20% test sets** using stratification on the target (random_state = 42). Protected attributes are re-aligned with the same train/test indices for fairness evaluation.

Console output: Train: 5,771 | Test: 1,443

4. Classification Models

Three classification models are trained in models.py:

- **Logistic Regression:** LogisticRegression(max_iter=1000, random_state=42), used as an interpretable linear baseline.
- **Random Forest** — RandomForestClassifier(n_estimators=100, random_state=42), an ensemble of decision trees.
- **Gradient Boosting** — sklearn.ensemble.GradientBoostingClassifier(n_estimators=100, random_state=42), a boosted ensemble of decision trees.

Each model is trained on the training split, evaluated on the test split using class predictions, probabilities, accuracy, and ROC-AUC, then saved as results/*.pkl for reuse without needing to retrain.

5. Model Performance Comparison

evaluation.py reloads each saved model and, on the shared test set (n = 1,443), produces a confusion matrix, ROC curve, and precision–recall curve. The table below consolidates the metrics that can be read directly from the generated figures for all three models.

5.1 Model Comparison Summary

Model	Accuracy	ROC-AUC	PR-AUC	Rank
Gradient Boosting	0.6910	0.7443	0.7033	1 st the best overall
Logistic Regression	0.6834	0.7369	0.6930	2 nd
Random Forest	0.6396	0.6837	0.6209	3 rd
Comparison between the models	- Gradient Boosting is the best performing model on every metric (Accuracy, ROC-AUC, PR-AUC). - Logistic Regression is a close second, within ~1 point of accuracy and ~0.01 AUC of Gradient Boosting. - Random Forest trails with the lowest accuracy (0.6396) and ROC-AUC (0.6837)			

5.2 Gradient Boosting – Confusion Matrix, Roc and Precision- Recall

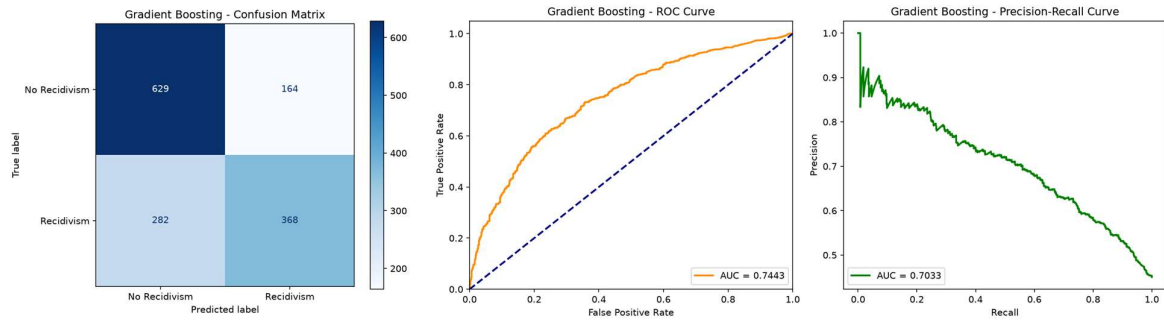


Figure 1. Gradient Boosting:

confusion matrix (left), ROC curve AUC = 0.7443 (center), precision recall curve AUC = 0.7033 (right)

5.3 Logistic Regression – Confusion Matrix, ROC and Precision-Recall

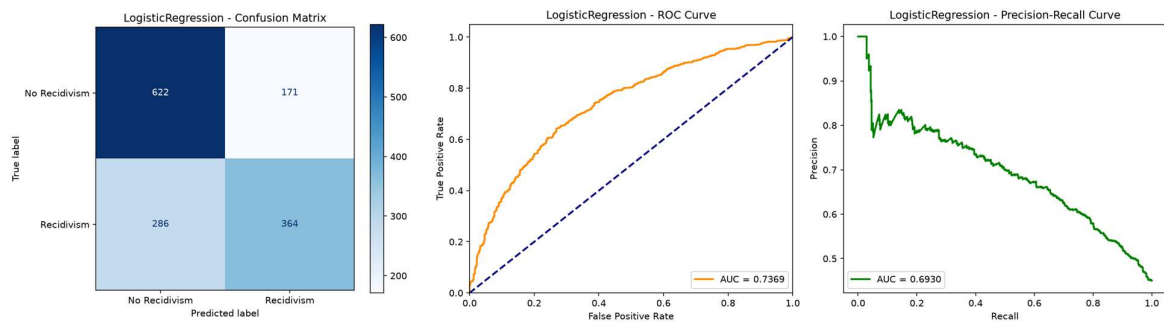


Figure 2. Logistic Regression:

confusion matrix (left), ROC curve AUC = 0.7369 (center), precision recall curve AUC = 0.6930 (right)

5.4 Random Forest – Confusion Matrix, ROC and Precision-Recall

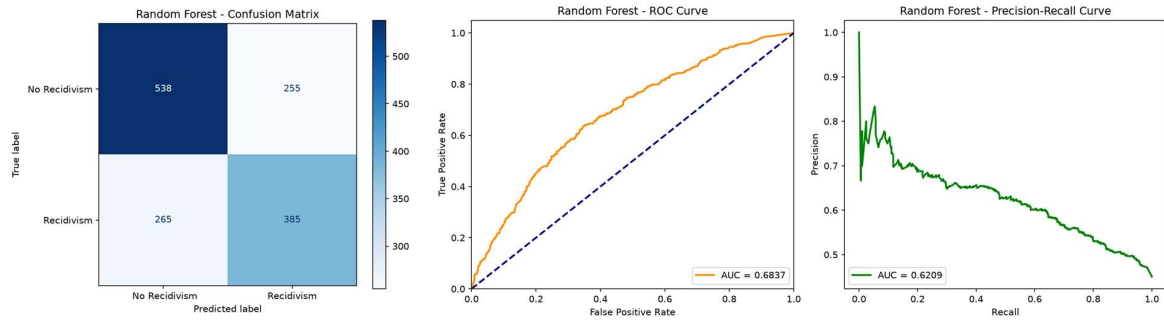


Figure 3. Random Forest:

confusion matrix (left), ROC curve AUC = 0.6837 (center), precision recall curve AUC = 0.6209 (right)

5.5 Discussion

Gradient Boosting performs slightly better than Logistic Regression in accuracy, ROC-AUC, and PR-AUC, but the difference is small. Random Forest trails both, with the lowest accuracy (0.6396) and ROC-AUC (0.6837) of the three models. Random Forest does miss fewer recidivists in absolute terms (265 of 650) than Gradient Boosting (282) or Logistic Regression (286), but this comes with a much higher false-positive count (255 non-recidivists incorrectly flagged), so it is not a straightforward improvement.

Overall, no model is clearly better calibrated, and the six-feature set likely limits prediction performance due to the noisy nature of COMPAS recidivism prediction.

6. Explainability (SHAP and LIME)

explainability.py generates both a global (SHAP) explanation and a local (LIME) explanation for an individual prediction using the Gradient Boosting model.

6.1 SHAP (Global Feature Importance)

Uses shap.TreeExplainer for the Gradient Boosting model to calculate SHAP values on the test set and display them in a beeswarm-style summary plot.

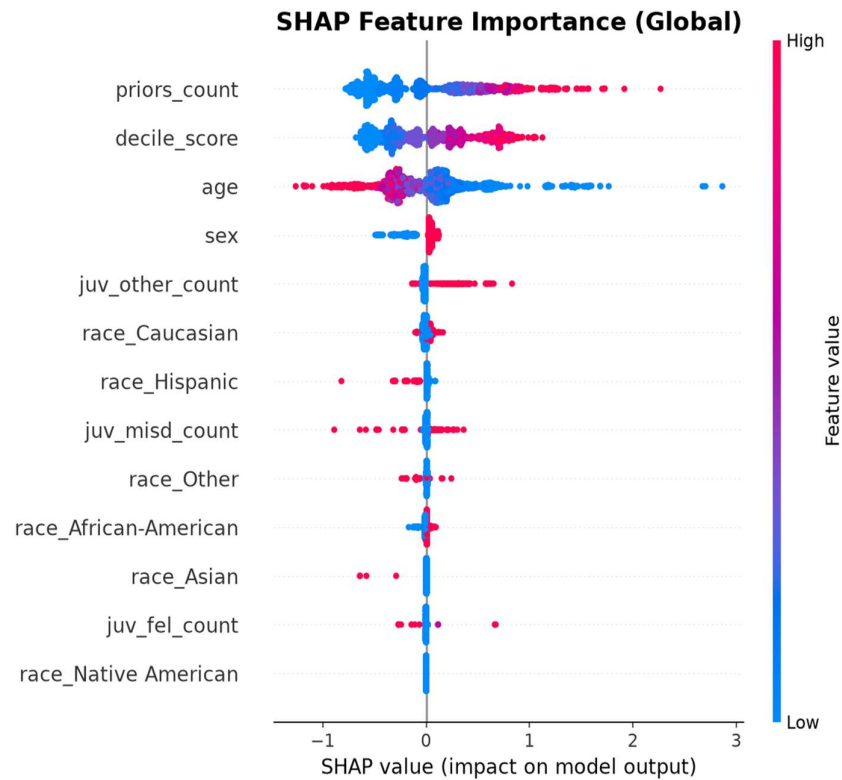


Figure 4. SHAP global feature

importance summary for the Gradient Boosting model.

priors_count and decile_score are the most influential features, with higher values pushing predictions toward recidivism. age has a strong negative effect, with older age pushing predictions toward no recidivism. Sex and one-hot encoded race features have smaller SHAP effects, but their use as model inputs is still important to note in a fairness-sensitive context.

6.2 LIME(Local Explanation)

The LIME fits on `x_train`, the same distribution used to train the model. This is the correct approach because LIME perturbs samples around the reference distribution and should not use held-out test data.

With this fix, LimeTabularExplainer is fit on the training features and used to explain one test prediction (instance 0), showing the ten features with the largest local contributions.

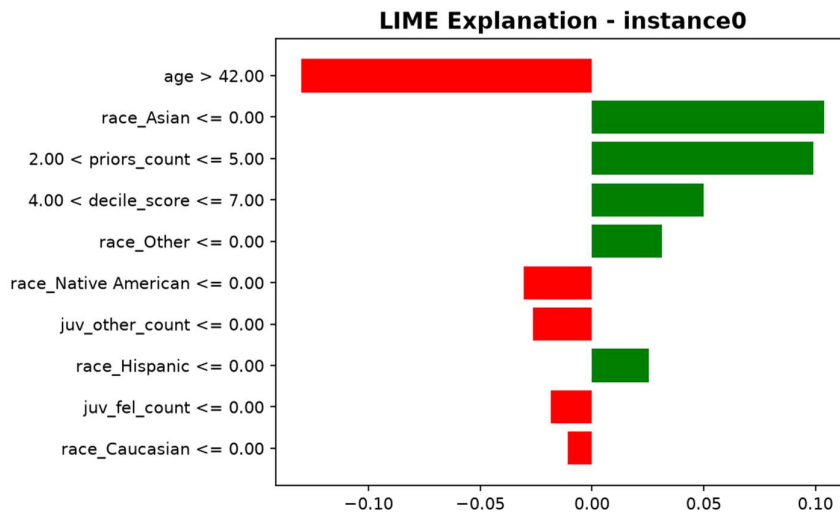


Figure 5. LIME local explanation

for test instance 0 (Gradient Boosting model).

For this instance, **age > 42** is the strongest factor pushing the prediction toward **no recidivism**. In contrast, **race_Asian ≤ 0**, **priors_count** between 2–5, and **decile_score** between 4–7 push the prediction toward **recidivism**. Several race indicators also appear among the most influential local features, which is important for the fairness analysis

7. Fairness Evaluation

fairness.py computes, for each trained model and each protected attribute (race, sex), the group-wise False Positive Rate (FPR), False Negative Rate (FNR), Precision and Recall, plus a disparity summary equal to the maximum-minus-minimum value of each metric across groups. The results below, captured from the console output, cover all three trained models evaluated on the 1,443 row test set.

7.1 Fairness by Race (Gradient Boosting)

Group	Count	FPR	FNR	Precision	Recall
African-American	736	0.2901	0.3102	0.7107	0.6898
Asian	4	0.0000	1.0000	0.0000	0.0000
Caucasian	486	0.1498	0.5866	0.6167	0.4134
Hispanic	126	0.1389	0.6667	0.6429	0.3333
Native American	5	0.0000	0.0000	1.0000	1.0000
Other	86	0.0638	0.5897	0.8421	0.4103

Race disparities (max – min across groups): FPR = 0.2901, FNR = 1.0000, Precision = 1.0000, Recall = 1.0000

African-American defendants (n = 736) have an FPR of 0.2901, nearly twice the 0.1498 FPR for Caucasian defendants (n = 486). This means African-American defendants who did not reoffend were more likely to be incorrectly flagged as high-risk. In contrast, the FNR is higher for Caucasian defendants (0.5866) than for African-American defendants (0.3102), meaning more Caucasian recidivists were missed.

The extreme disparity values for FNR, Precision, and Recall are mainly caused by the very small Asian (n = 4) and Native American (n = 5) groups, so these results should be interpreted with caution.

7.2 Fairness by Sex – Gradient Boosting

Group	Count	FPR	FNR	Precision	Recall
Female	291	0.0909	0.6442	0.6852	0.3558
Male	1152	0.2426	0.3938	0.6925	0.6062

Sex disparities (max – min): FPR = 0.1517, FNR = 0.2504, Precision = 0.0073, Recall = 0.2504

Precision is almost the same for females (0.6852) and males (0.6925). However, males have a higher FPR (0.2426 vs 0.0909), meaning they were more likely to be incorrectly flagged as high-risk. Females have a higher FNR (0.6442 vs 0.3938), meaning the model missed more female recidivists.

7.2 Fairness by Race and Sex – Logistic Regression

Race

Group	Count	FPR	FNR	Precision	Recall
African-American	736	0.3094	0.3289	0.6915	0.6711
Asian	4	0.0000	1.0000	0.0000	0.0000
Caucasian	486	0.1498	0.5587	0.6320	0.4413
Hispanic	126	0.1528	0.6111	0.6562	0.3889
Native American	5	0.0000	0.5000	1.0000	0.5000
Other	86	0.0426	0.6923	0.8571	0.3077

Race disparities (max – min across groups): FPR = 0.3094, FNR = 0.6711, Precision = 1.0000, Recall = 0.6711

As with Gradient Boosting, African-American defendants (n = 736) have a higher FPR (0.3094) than Caucasian defendants (n = 486, FPR = 0.1498), roughly twice as high. The FNR is again higher for Caucasian defendants (0.5587) than for African-American defendants (0.3289), meaning more Caucasian recidivists were missed by Logistic Regression as well.

Sex

Group	Count	FPR	FNR	Precision	Recall
Female	291	0.0695	0.7500	0.6667	0.2500
Male	1152	0.2607	0.3810	0.6815	0.6190

Sex disparities (max – min): FPR = 0.1912, FNR = 0.3690, Precision = 0.0148, Recall = 0.3690

Males have a higher FPR under Logistic Regression (0.2607 vs 0.0695 for females), consistent with the Gradient Boosting pattern. The FNR gap by sex is wider here than for Gradient Boosting: Logistic Regression misses 75.00% of female recidivists (FNR = 0.7500) compared with 38.10% of male recidivists (FNR = 0.3810).

7.3 Fairness by Race and Sex – Random Forest

Race

Group	Count	FPR	FNR	Precision	Recall
African-American	736	0.3867	0.3717	0.6267	0.6283
Asian	4	0.0000	0.5000	1.0000	0.5000
Caucasian	486	0.2964	0.4413	0.5236	0.5587
Hispanic	126	0.2778	0.5370	0.5556	0.4630
Native American	5	0.3333	0.0000	0.6667	1.0000
Other	86	0.0638	0.4359	0.8800	0.5641

Race disparities (max – min across groups): FPR = 0.3867, FNR = 0.5370, Precision = 0.4764, Recall = 0.5370.

Random Forest has the highest FPR across most groups. African-American defendants have an FPR of 0.3867 compared with 0.2964 for Caucasian defendants. The FNR is also higher for Caucasian defendants (0.4413) than for African-American defendants (0.3717), following the same pattern as the other models.

Sex

Group	Count	FPR	FNR	Precision	Recall
Female	291	0.1925	0.5577	0.5610	0.4423
Male	1152	0.3614	0.3791	0.6075	0.6209

Sex disparities (max – min): FPR = 0.1689, FNR = 0.1786, Precision = 0.0465, Recall = 0.1786

Males have a higher FPR than females (0.3614 vs 0.1925), while the model misses more female recidivists (FNR = 0.5577 vs 0.3791). The sex disparities are smaller than the race disparities for Random Forest.

7.4 Fairness Visualization

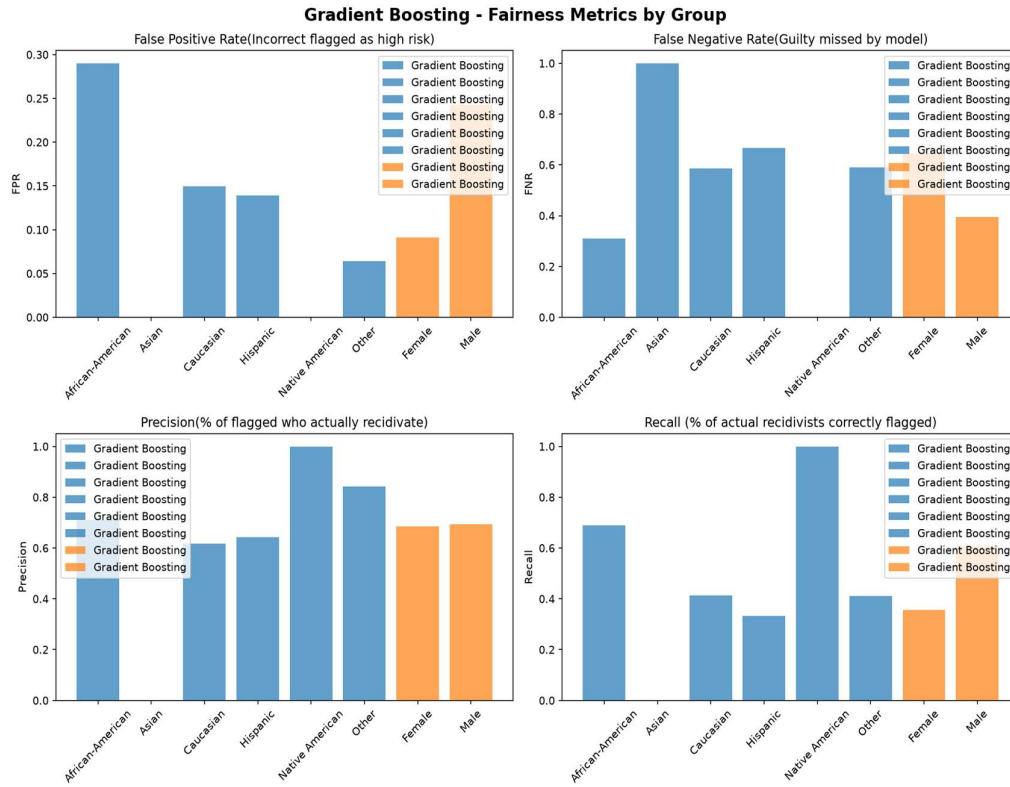


Figure 6. Gradient Boosting

FPR, FNR, Precision and Recall by race and by sex subgroup.

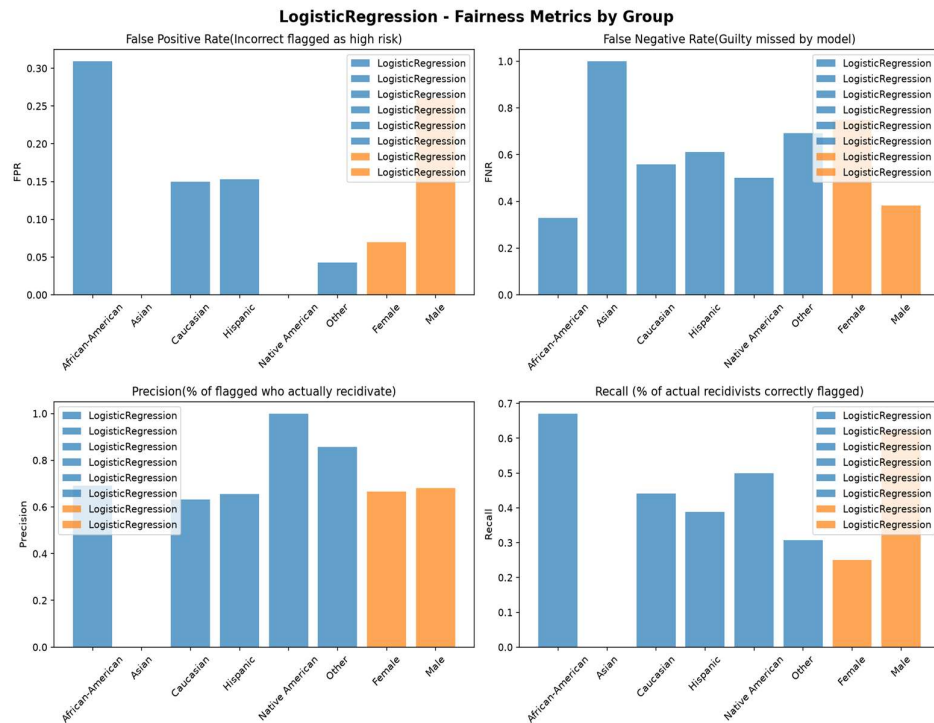


Figure 7. Logistic Regression

FPR, FNR, Precision and Recall by race and by sex subgroup.

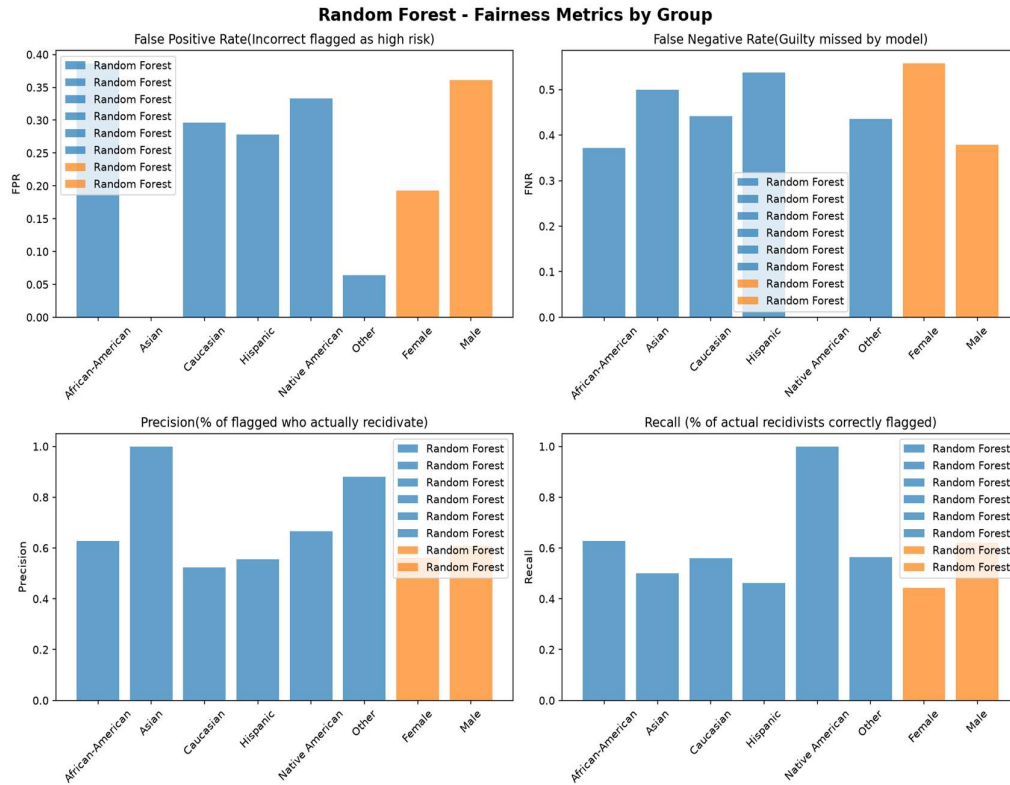


Figure 8. Random Forest

FPR, FNR, Precision and Recall by race and by sex subgroup.

8. Discussion and Limitations

- **Small feature set:** Only six features are used for modelling, which may limit predictive performance (ROC-AUC $\approx$ 0.74).
- **decile_score as a feature:** Since decile_score comes from the original COMPAS tool, using it may cause the models to reproduce some of its behaviour.
- **Race as a model feature:** Race is included as a predictive feature, which raises fairness concerns because it is a protected attribute.
- **Small groups:** The Asian ($n = 4$) and Native American ($n = 5$) groups are too small for reliable fairness results. The main comparison should focus on the larger African-American and Caucasian groups.
- **Model selection trade-off:** Random Forest has the lowest accuracy and ROC-AUC of the three models and the highest false-positive rates across most demographic groups, so it is not a clear choice over Gradient Boosting or Logistic Regression despite missing slightly fewer recidivists in absolute terms.
- **Fairness metrics:** The analysis uses FPR, FNR, precision, and recall but does not include calibration or statistical-parity measures, which are also important in fairness analysis.

9. Conclusion

The project demonstrates a complete modular workflow for COMPAS recidivism prediction. Three models were trained, evaluated, and compared, with SHAP and LIME used for explanations and fairness metrics calculated for race and sex across all three models. Gradient Boosting and Logistic Regression achieved similar performance (ROC-AUC $\approx$ 0.74), while Random Forest trailed behind (ROC-AUC = 0.6837) and produced the highest false-positive rates. The fairness results showed clear and consistent differences between groups across all three models, with African-American defendants having a higher FPR and Caucasian defendants a higher FNR. There were also sex differences, with men more often over-flagged and

women more often under-flagged, most pronounced for Logistic Regression. These results show the importance of considering both model performance and fairness when using COMPAS data.

10. How to Run the Project

The project is intended to be run as a package of sequential stages from the project root, with a `place_compas.csv` file expected under `data/compas.csv` (as read by `src/data_loader.py`).

Step	Command	Produces
1. Install dependencies	<code>pip install -r requirements.txt</code>	Required Python packages: pandas, scikit-learn, shap, lime, matplotlib, etc
2. Train models	<code>python -m src.models</code>	<code>results/*.pkl</code> (LogisticRegression, Random Forest, Gradient Boosting)
3. Evaluate models	<code>python -m src.evaluation</code>	Evaluation plots (confusion matrix, ROC, PR curves)
4. Explain predictions	<code>python -m src.explainability</code>	SHAP and LIME outputs
5. Evaluate fairness	<code>python -m src.fairness</code>	Fairness tables and charts

The dataset should be placed at `data/compas.csv`. Each stage uses the results from the previous stages.